



Banco de Dados Processamento de Transações

Marcel Oliveira

1



Transação

- Conceito que fornece mecanismos para descrição de unidades lógicas de processamento de dados
- Sistemas de processamento de transações
 - Reserva de passagens
 - Bancos
 - Cartões de crédito
 - Ações
 - Compras on-line

2



Transação

- Sistemas de processamento de transações
 - Alta disponibilidade
 - Baixo tempo de resposta
 - Usuários concorrentes
- Extrema utilidade

3



Nesta Aula...

- Introdução ao processamento de transações
- Conceitos de transação e sistemas BD
- Propriedades desejáveis de transações
- Caracterização de plano de execução baseado na restaurabilidade
- Caracterização de plano de execução baseado na seriabilidade
- Suporte para transações em SQL

4



Transações

- Unidade lógica de processamento de BD que inclui um ou mais acessos
 - Leitura
 - Escrita (inserção ou atualização)
 - Remoção
- Uma transação (conjunto de operações) pode ser especificada em uma linguagem de alto-nível como SQL e submetida interativamente, ou pode fazer parte de um programa
- Delimitadores da transação
- Uma aplicação pode conter várias transações separadas pelos delimitadores de transação

5



Modelo Simples de BD

- Nesta aula, utilizaremos o seguinte modelo (simplificado) de BD
- BD é uma coleção de itens de dados com nomes
- Granularidade do dado
 - Um campo, um registro, bloco de disco
 - Conceitos independem da granularidade
- Operações básicas
 - *read_item(X)*: lê um item do BD chamado *X* em uma variável de programa (por simplificação, também chamada de *X*)
 - *write_item(X)*: escreve o valor de uma variável de programa *X* no item do BD chamado *X*

6



Modelo Simples de BD

- Passos do *read_item(X)*
 - Encontre o endereço do bloco de disco que contém o item X
 - Copie o bloco de disco em um buffer na memória principal (se ele já não estiver no buffer)
 - Copie o item X do buffer para a variável de programa X

7



Modelo Simples de BD

- Passos do *write_item(X)*
 - Encontre o endereço do bloco de disco que contém o item X
 - Copie este bloco de disco em um buffer na memória principal (se ele já não estiver em um buffer)
 - Copie o item X da variável de programa de nome X para a localização correta no buffer
 - Armazene o bloco atualizado do buffer de volta no disco (imediatamente ou em algum momento posterior)
 - Quando? Administrador do SGBD + SO

8



Exemplos de Transação

(a)

T_1

```
read_item (X);  
X:=X-N;  
write_item (X);  
read_item (Y);  
Y:=Y+N;  
write_item (Y);
```

Conjunto de leitura = { X, Y }
Conjunto de escrita = { X, Y }

(b)

T_2

```
read_item (X);  
X:=X+M;  
write_item (X);
```

Conjunto de leitura = { X }
Conjunto de escrita = { X }

9



Por que Controle de Concorrência?

- Duas transações como T_1 e T_2 podem ser executadas por diferentes usuários concorrentemente
 - Acesso ao mesmo item do BD
 - Descontrole pode levar à inconsistência

10



Por que Controle de Concorrência?

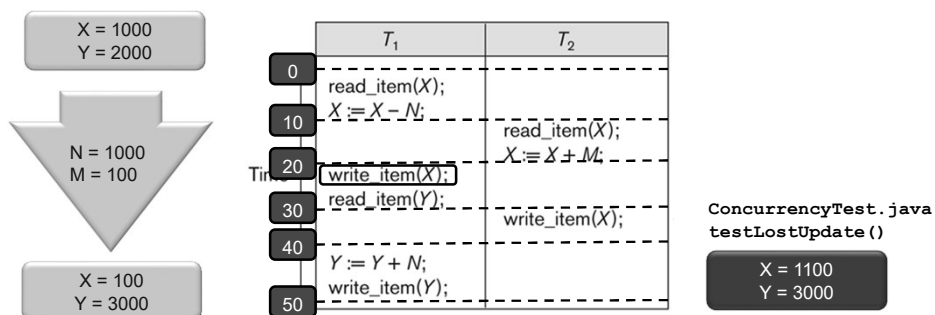
- Problemas
 - Atualização perdida
 - Atualização temporária (leitura suja)
 - Sumário incorreto

11



Problema da Atualização Perdida

- Ocorre quando duas transações que acessam os mesmos itens de BD têm suas operações intercaladas de uma maneira que uma sobreescreve a escrita da outra



12



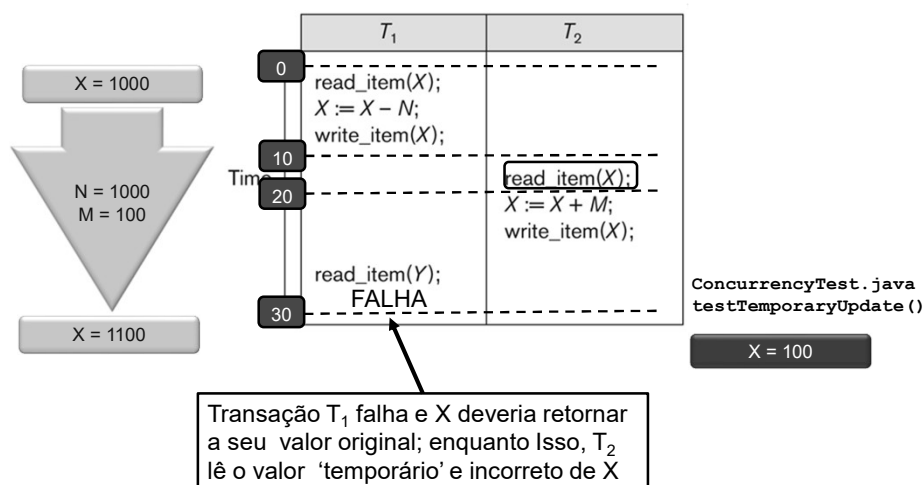
Problema da Atualização Temporária

- Ocorre quando uma transação atualiza um item do BD e então falha por alguma razão
- O item atualizado é acessado por outra transação antes que o seu valor retorne ao valor original

13



Problema da Atualização Temporária



14



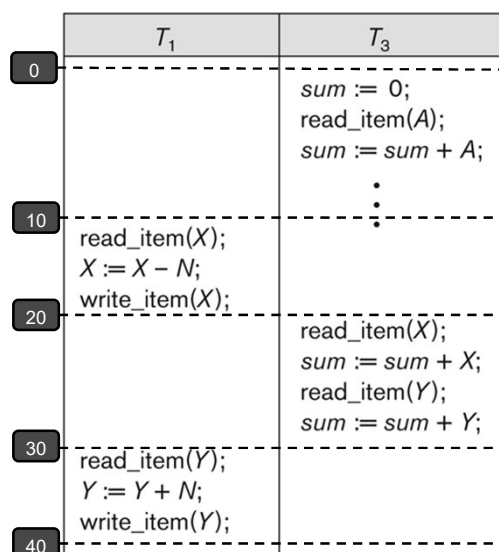
Problema do Sumário Incorreto

- Se uma transação está calculando uma função agregada sobre um número de itens enquanto outras transações estão atualizando alguns destes itens
- A função agregada pode calcular alguns valores antes deles serem atualizados e outros depois deles serem atualizados

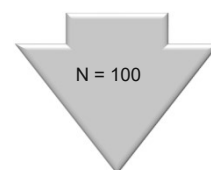
15



Problema do Sumário Incorreto



X = 1000
Y = 2000



X = 900
Y = 2100
Sum = 3000

Sum = 2900

ConcurrencyTest.java
testIncorrectSummary()

16



Por que a Restauração é Necessária?

- Na execução de uma transação o SGBD deve garantir
 - Todas as operações foram bem sucedidas e o resultado será gravado no BD
- Caso contrário
 - A transação não terá nenhum efeito no BD
- Falhas de transação...

17



Falhas de Transação

- Falhas de transação
 - Falha de computador
 - Problema de hardware ou software durante a execução da transação
 - Erro de transação ou de sistema
 - Operações da transação podem levá-la a falhas (overflow, divisão por zero, parâmetros errados, lógica de programa)
 - Usuário cancela a execução

18



Falhas de Transação

- Erros locais ou condições de exceção detectadas pela transação
 - Certas condições necessitam do cancelamento da transação
 - Falta de dados
 - Saldo insuficiente
 - Programa aborta
- Imposição do controle de concorrência
 - O método de controle de concorrência pode decidir abortar a transação, para ser reiniciada mais tarde, porque ela viola a serialização ou porque muitas transações estão em um estado de deadlock

19



Falhas de Transação

- Falha de disco
 - Mal funcionamento de leitura ou escrita, ou um *crash* no cabeçote de leitura e escrita podem levar à perda dos dados de blocos do disco
- Problemas físicos e catastróficos
 - Lista sem fim de problemas
- Essas duas falhas são mais “raras” que as outras

20



Conceitos de Transação

- Uma transação é uma unidade atômica de trabalho que ou é completada por inteiro ou não é realizada
 - A fim de poder restaurar, o sistema precisa manter o controle de quando a transação inicia (*start*), termina (*terminate*), comita (*commit*), e aborta (*abort*)

21



Conceitos de Transação

- Gerente de restauração mantém as seguintes informações
 - ***begin_transaction***: Marca o início da execução de uma transação
 - ***read*** ou ***write***: Marcam operações de leitura e escrita em itens do BD que são executadas como parte de uma transação
 - ***end_transaction***: Marca o fim das operações de leitura e escrita e, conseqüentemente, o limite final da execução da transação
 - Verificar se as alterações podem ser efetivamente feitas no BD ou se ela deve ser abortada por violar controle de concorrência ou outros motivos

22



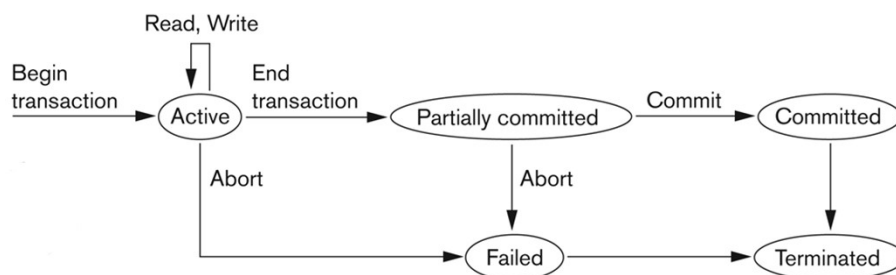
Conceitos de Transação

- Gerente de restauração mantém as seguintes informações
 - ***commit_transaction***: indica término com sucesso de uma transação de modo que todas as suas alterações persistiram no BD
 - ***rollback*** (ou ***abort***): indica que a transação terminou sem sucesso, e que quaisquer alterações feitas por esta transação no BD devem ser desfeitas

23



Estados de uma Transação



24



O Log do Sistema

- Mantém as informações de todas as operações das transações que afetam valores dos itens do BD
 - Esta informação pode ser necessária para permitir a restauração de falhas de transação
- Mantido no disco de forma a não ser atingido por outra falha que não a de disco ou catastrófica
- Periodicamente feito um *backup* de modo a suportar falhas de disco e catastróficas

25



O Log do Sistema

- Seja T um identificador único de transação gerado automaticamente pelo sistema
- Tipos de registro de *log*
 - $[start_transaction, T]$
 - $[write_item, T, X, old_value, new_value]$
 - $[read_item, T, X]$
 - $[commit, T]$
 - $[abort, T]$

26



O Log do Sistema

- Não necessariamente temos todos esses tipos
 - Protocolos para restauração que evitam o rollback em cascata não exigem que as operações de leitura sejam gravadas no log
 - Outros protocolos exigem entradas de escrita mais simples sem o novo valor

27



O Log do Sistema

- Restauração usando os registro do *log*
 - Se o sistema falhar, podemos restaurar o BD para um estado consistente examinando o log e usando técnicas de recuperação de BD (Capítulo 23)
 - De uma maneira geral
 - Operações de escrita podem ser desfeitas (operação *undo*) analisando o log *backwards* e restaurando o valor antigo do item
 - Operações de escrita podem ser refeitas (operação *redo*) analisando o log *forward* e restaurando o valor novo do item

28



Ponto de Efetivação de uma Transação

- *Commit Point* de uma transação
 - Todas as operações de acesso ao BD foram executadas com sucesso e seus efeitos registrados no *log*
 - Além desse ponto, assumimos que a transação está *committed* e seu efeito está permanentemente registrado no BD
 - A transação escreve um $[commit, T]$ no *log*
- *Roll Back* de uma transação
 - Necessária para transações que têm um $[start_transaction, T]$ no *log* mas não têm um $[commit, T]$

29



Ponto de Efetivação de uma Transação

- Forçando a escrita no *log*
 - Transações que escreveram entradas *commit* no *log* também devem ter escrito todas as operações de escrita no *log*
 - Antes de uma transação chegar no ponto de efetivação, todas as entradas do *log* devem ser passadas do *buffer* para o disco
- Refazendo transações
 - No caso de um *crash* de sistema, apenas as entradas do *log* que foram escritas no disco são consideradas

30



Propriedades ACID

- **Atomicidade**
 - Uma transação é uma unidade atômica de processamento (ou tudo ou nada)
- **Preservação de Consistência**
 - Uma execução correta de uma transação leva o BD de um estado consistente para um outro estado também consistente
- **Isolamento**
 - As atualizações de uma transação não deveriam ser percebidas por outras transações antes de seu *commit*
 - Resolve o problema da atualização temporária
- **Durabilidade**
 - Após o *commit*, nenhuma alteração feita por uma transação deve ser perdida devido a uma falha

31



Plano de Execução

- A ordem de execução de operações de transações concorrentes é chamada de ***Plano de Execução (Schedule)***
 - $T_1, \dots, T_n$ - Transações
 - S - Plano de Execução
 - Ordem em que as operações das diversas transações acontecem
 - As operações de cada transação T_i devem acontecer em S na mesma ordem em que elas acontecem em T_i
 - Operações de outras transações podem ser intercaladas

32



Plano de Execução

1	2	3	4
---	---	---	---

1	2	3	4
---	---	---	---

1	2	3	4
---	---	---	---

1	1	2	1	3	2	3	4	2	3	4	4
---	---	---	---	---	---	---	---	---	---	---	---

33



Plano de Execução

- Operações
 - r – read
 - w – write
 - c – commit
 - a – abort
 - Exemplo
 - $r_1(X)$ – leitura do item X pela transação T_1

34



Plano de Execução

T_1	T_2
$\text{read_item}(X);$ $X := X - N;$ $\text{write_item}(X);$ $\text{read_item}(Y);$ $Y := Y + N;$ $\text{write_item}(Y);$	$\text{read_item}(X);$ $X := X + M;$ $\text{write_item}(X);$

$S_a: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y);$

35



Plano de Execução

	T_1	T_2
Time ↓	<pre>read_item(X); X := X - N; write_item(X);</pre>	<pre>read_item(X); X := X + M; write_item(X);</pre>
	<pre>read_item(Y); FALHA</pre>	

$S_b: r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); a_1;$

36



Plano de Execução

- **Conflito de operações**

- Pertencem a transações diferentes
- Acessam o mesmo item
- Pelo menos uma das operações é um *write_item*

$S_a: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_2(Y);$



37



Plano de Execução

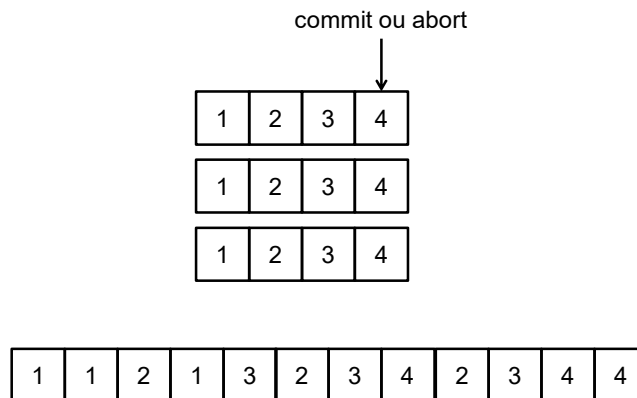
- **Plano completo** S de transações $T_1, \dots, T_n$

- Operações em S são exatamente as de $T_1, \dots, T_n$ tendo um commit ou um abort como última operação de $T_1, \dots, T_n$ no plano
- A ordem das operações de cada transação T_i no plano respeita a ordem das operações em T_i

38



Plano de Execução



39



Plano de Execução

- **Plano completo** S de transações $T_1, \dots, T_n$
 - Para quaisquer duas operações conflitantes, uma das duas precisa aparecer antes da outra no plano

40



Plano de Execução

- Em geral, é difícil ter um plano completo porque novas transações estão sempre sendo submetidas
- Projeção de efetivadas $C(S)$ de um plano S
 - Inclui em S apenas as transações efetivadas

41



Caracterizando Planos de Execução

- Em alguns planos a restauração é fácil e em outros bastante complicada
- É importante caracterizar os tipos de planos nos quais a restauração é possível (e/ou simples)

42



Caracterizando Planos

- Restaurabilidade
- Seriabilidade

43



Caracterizando Planos baseados na Restaurabilidade

- Plano restaurável
 - Nenhuma transação precisa ser desfeita (*roll back*)
 - Um plano de execução S é restaurável se nenhuma transação T em S for efetivada (*commit*) antes que todas as transações T' que escrevem um item que T leia tenham também sido efetivadas
- Planos não-restauráveis não deveriam ser admitidos

44



Caracterizando Planos baseados na Restaurabilidade

- Que planos são restauráveis?

$S_c: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); c_2; a_1;$ 

$S_d: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); c_1; c_2;$ 

$S_e: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); a_1; a_2;$ 

Um plano de execução S é restaurável se nenhuma transação T em S for efetivada (*commit*) antes que todas as transações T' que escrevem um item que T leia tenham também sido efetivadas

45



Caracterizando Planos baseados na Restaurabilidade

- Em um plano restaurável, nenhuma transação efetivada deveria precisar ser revertida
- Porém, é possível reversão em cascata
 - Uma transação não-efetivada tem que ser revertida porque leu um item de uma transação que falhou

$S_e: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); a_1; a_2;$ 

46



Caracterizando Planos baseados na Restaurabilidade

- Reversão em cascata
 - Pode consumir muito tempo
- ***Plano livre de reversão em cascata***
 - Cada transação lê apenas itens que são escritos por transações efetivadas
 - Todos os itens lidos já foram efetivados, logo não precisarão ser revertidos

47



Caracterizando Planos baseados na Restaurabilidade

- ***Plano livre de reversão em cascata***

$$\begin{array}{l} S_d: r_1(X); w_1(X); r_2(X); r_1(Y); w_2(X); w_1(Y); c_1; c_2; \\ \downarrow \\ S_d: r_1(X); w_1(X); r_1(Y); w_1(Y); c_1; r_2(X); w_2(X); c_2; \end{array}$$

48



Caracterizando Planos baseados na Restaurabilidade

- **Plano restrito**

- Transações não podem nem ler nem gravar um item X até que a última transação que escreveu em X tenha sido efetivada
- Uma operação pode ser desfeita simplesmente restaurando o valor antigo do item
 - Pode não funcionar em planos restauráveis ou livres de reversão em cascata (exemplo no livro)

49



Caracterizando Planos baseados na Restaurabilidade

- **Resumo**

- Restaurabilidade
- Livre de reversão em cascata
- Restrito

+ Concorrência
- Eficiência de Restauração
↑↓
- Concorrência
+ Eficiência de Restauração

- **Implicação**

- Todo plano restrito é livre de reversão em cascata
- Todo plano livre de reversão em cascata é restaurável

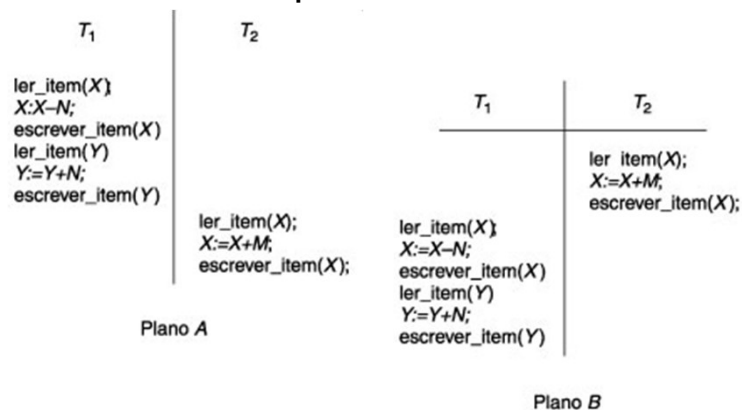
FIM DA AULA 01

50



Caracterizando Planos baseados na Seriabilidade

- Sem intercalação de transações temos duas possibilidades

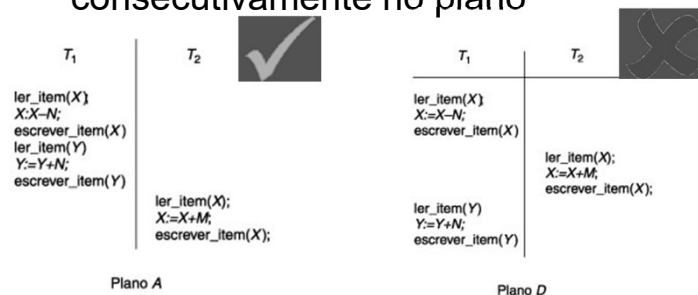


51



Caracterizando Planos baseados na Seriabilidade

- Plano de execução serial
 - Para toda transação T que participa do plano de execução, todas as operações de T são executadas consecutivamente no plano



52



Caracterizando Planos baseados na Seriabilidade

- Plano de execução serial
 - Apenas uma transação está ativa por vez
 - Não há intercalação
 - Cada plano de execução é considerado correto (baseado no princípio da conservação da consistência)
 - Limitam a concorrência
 - Geralmente, inaceitáveis na prática

53



Caracterizando Planos baseados na Seriabilidade

- Com intercalação de transações temos várias possibilidades

T_1	T_2	T_1	T_2
ler_item(X); $X := X - N;$		ler_item(X); $X := X - N;$ escrever_item(X)	
	ler_item(X); $X := X + M;$		ler_item(X); $X := X + M;$ escrever_item(X);
escrever_item(X) ler_item(Y)		ler_item(Y) $Y := Y + N;$ escrever_item(Y)	
$Y := Y + N;$ escrever_item(Y)	escrever_item(X);		
Plano C		Plano D	

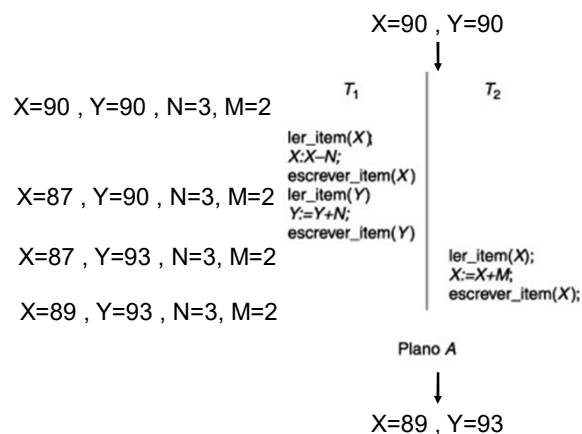
Quais delas são corretas??? Para isso usamos o conceito de seriabilidade de planos de execução

54



Caracterizando Planos baseados na Seriabilidade

- Analisemos os planos de execução

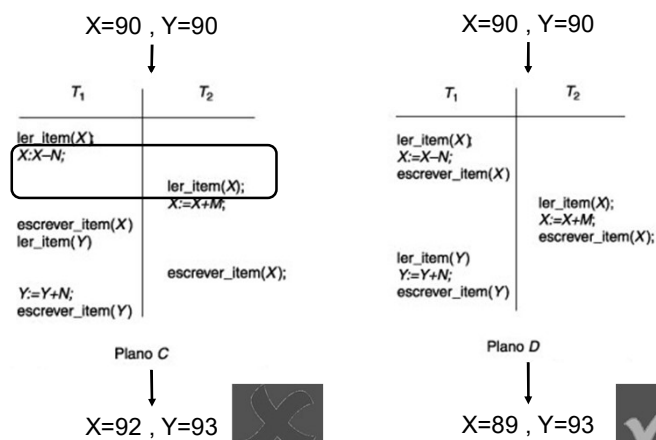


55



Caracterizando Planos baseados na Seriabilidade

- Analisemos os planos de execução



56



Caracterizando Planos baseados na Seriabilidade

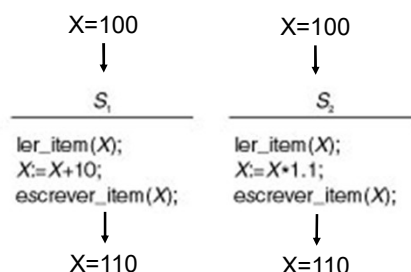
- Que planos não-seriais sempre fornecem resultados corretos? Que planos podem fornecer resultados errôneos?
- Seriabilidade
 - Um plano de execução é serializável se ele é equivalente a algum plano de execução serial nas mesmas transações

57



Caracterizando Planos baseados na Seriabilidade

- Equivalência de planos
 - Resultados equivalentes?



Não são equivalentes!!!
Equivalência de conflito e Equivalência de Visão

58



Caracterizando Planos baseados na Seriabilidade

- Equivalência de conflito
 - Dois planos são conflito equivalentes se a ordem de quaisquer duas operações conflitantes é a mesma em ambos os planos
 - Conflito de Operações
 - Exemplo
 - $S_1: \dots; r_1(X); \dots; w_2(X); \dots$
 - $S_2: \dots; w_2(X); \dots; r_1(X); \dots$



59



Caracterizando Planos baseados na Seriabilidade

- Plano de execução conflito serializável
 - Se ele for conflito equivalente a algum plano de execução serial
 - Podemos reordenar as operações não conflitantes em S até formar um plano de execução serial equivalente

60



Caracterizando Planos baseados na Seriabilidade

T_1	T_2	T_1	T_2
ler_item(X); $X:=X-N$; escrever_item(X) ler_item(Y) $Y:=Y+N$; escrever_item(Y)		ler_item(X); $X:=X-N$; escrever_item(X)	
	ler_item(X); $X:=X+M$; escrever_item(X);	ler_item(Y) $Y:=Y+N$; escrever_item(Y)	ler_item(X); $X:=X+M$; escrever_item(X);

Plano A

Plano D

$P_A: r_1(X); w_1(X); r_1(Y); w_1(Y); r_2(X); w_2(X);$ T_1 é a última a alterar Y

$P_D: r_1(X); w_1(X); r_2(X); w_2(X); r_1(Y); w_1(Y);$ T_2 é a última a alterar X

↑
Não conflitantes

$P_D: r_1(X); w_1(X); r_1(Y); w_1(Y); r_2(X); w_2(X);$

61



Caracterizando Planos baseados na Seriabilidade

T_1	T_2
ler_item(X); $X:=X-N$;	
escrever_item(X) ler_item(Y)	ler_item(X); $X:=X+M$;
$Y:=Y+N$; escrever_item(Y)	escrever_item(X);

Plano C



Não serializável

$P_A: r_1(X); w_1(X); r_1(Y); w_1(Y); r_2(X); w_2(X);$

$P_C: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y);$

$P_B: r_2(X); w_2(X); r_1(X); w_1(X); r_1(Y); w_1(Y);$

$P_C: r_1(X); r_2(X); w_1(X); r_1(Y); w_2(X); w_1(Y);$

62



Caracterizando Planos baseados na Seriabilidade

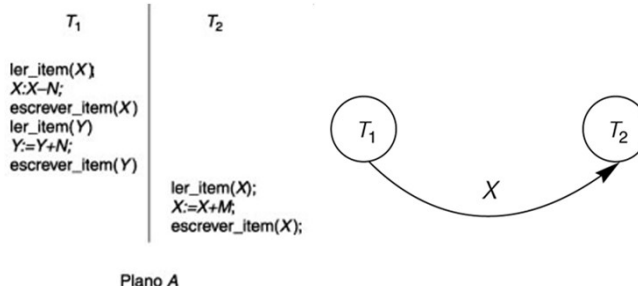
- Testando o conflito serialidade de um plano
 - Olha apenas para operações de leitura e escrita
 - Constrói um grafo de precedência (de serialização) com arestas direcionadas
 - Cada transação é um nó
 - Uma aresta é criada de T_i para T_j se uma das operações de T_i aparece antes de alguma operação conflitante de T_j
 - O plano de execução é serializável se, e somente se, o grafo não tiver ciclos

63



Caracterizando Planos baseados na Seriabilidade

- Testando o conflito serialidade de um plano



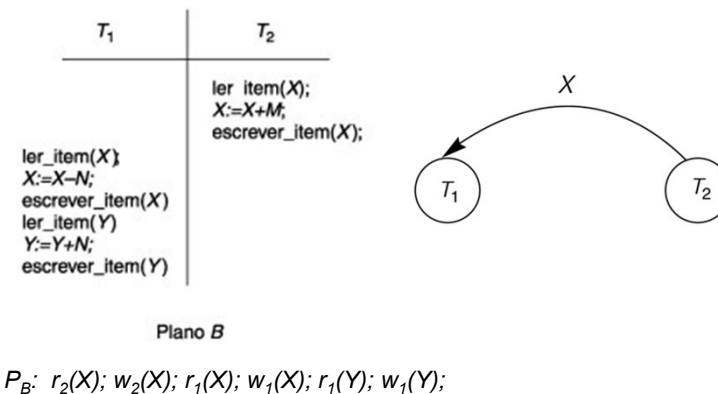
$P_A: r_1(X); w_1(X); r_1(Y); w_1(Y); r_2(X); w_2(X);$

64



Caracterizando Planos baseados na Seriabilidade

- Testando o conflito serialidade de um plano

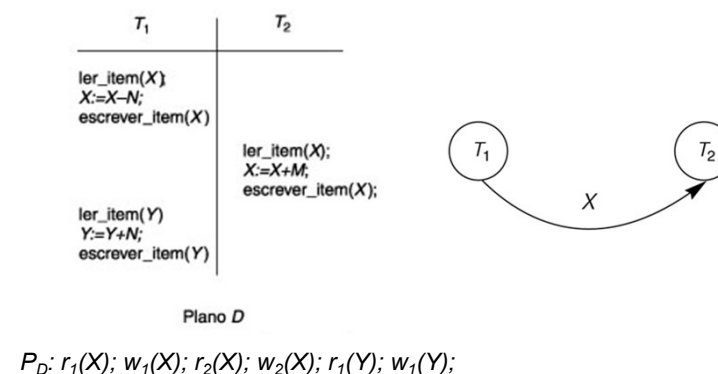


65



Caracterizando Planos baseados na Seriabilidade

- Testando o conflito serialidade de um plano

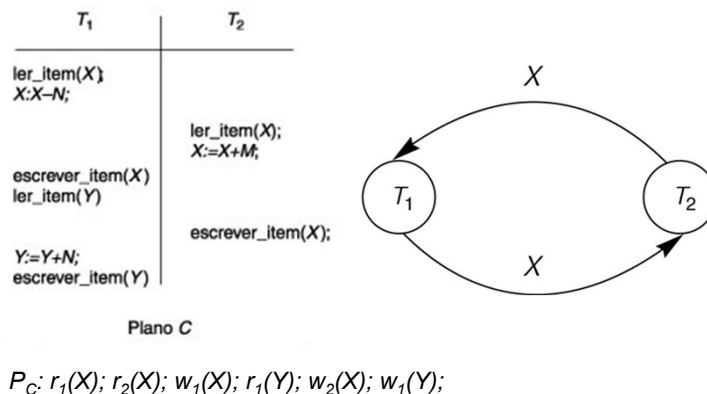


66



Caracterizando Planos baseados na Seriabilidade

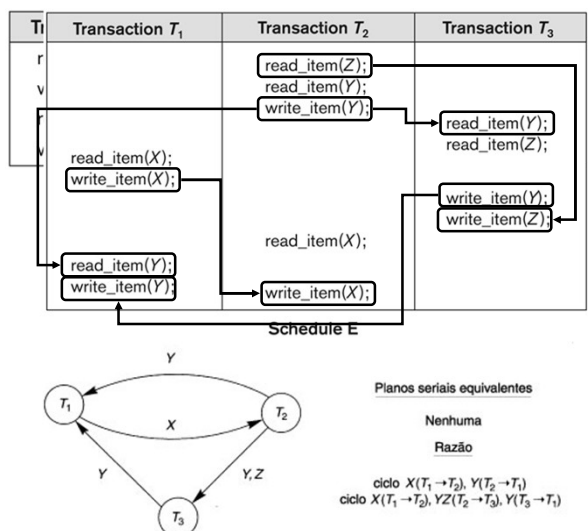
- Testando o conflito serialidade de um plano



67

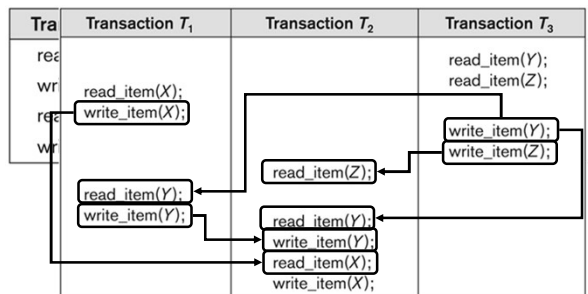


Caracterizando Planos baseados na Seriabilidade

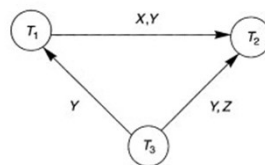


68

Caracterizando Planos baseados na Seriabilidade



Schedule F



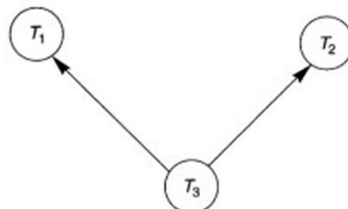
Planos seriais equivalentes

$T_3 \rightarrow T_1 \rightarrow T_2$

69

Caracterizando Planos baseados na Seriabilidade

- Pode haver mais de um plano serial equivalente



Planos seriais equivalentes

$T_3 \rightarrow T_1 \rightarrow T_2$

$T_3 \rightarrow T_2 \rightarrow T_1$

70



Caracterizando Planos baseados na Seriabilidade

- Ser serializável é diferente de ser serial
- Ser serializável implica que o plano de execução é correto
 - Deixará o BD em um estado consistente
 - A intercalação é apropriada e resultará em um estado como se as transações tivessem sido executadas serialmente
 - Foram, porém, executadas eficientemente (com concorrência)

71



Caracterizando Planos baseados na Seriabilidade

- Na prática, seriabilidade é difícil de verificar
 - Operações intercaladas ocorrem em um sistema operacional através de um agendamento
 - Difícil de determinar diante mão como as operações serão intercaladas
 - Impraticável cancelar o efeito de um plano caso verifiquemos que ele não é serializável

72



Caracterizando Planos baseados na Seriabilidade

- Abordagem prática
 - Uso de protocolos que garantem a seriabilidade de todas as suas transações
 - Não é possível determinar quando um agendamento começa e quando ele termina
 - Checamos apenas as operações de transações efetivadas

73



Caracterizando Planos baseados na Seriabilidade

- Abordagem atual na maioria dos SGBDs
 - Uso de bloqueios em duas fases (*two-phase locking*)
- Outros protocolos
 - Ordenação por marca de tempo (*timestamp*)
 - Multi-versões
 - Otimistas

74



Caracterizando Planos baseados na Seriabilidade

- Equivalência de visão
 - Definição menos restritiva de equivalência de planos
 - Enquanto cada operação de leitura de uma transação estiver lendo resultados de uma mesma operação de gravação em ambos os planos, as operações de gravação de cada transação precisam produzir os mesmos resultados (operações de leitura têm a mesma visão)

75



Caracterizando Planos baseados na Seriabilidade

- Dois planos S e S' são visão equivalentes se
 - O conjunto de transações participantes em S e S' sejam os mesmos e que S e S' contenham as mesmas operações dessas transações
 - Para qualquer operação $R_i(X)$ de T_i em S , se o valor de X lido pela operação for escrito por uma operação $W_j(X)$ de T_j (ou se for o valor original de X), a mesma condição deve valer para o valor de X lido pela operação $R_i(X)$ de T_i em S'
 - Se a operação $W_k(Y)$ de T_k é a última operação de escrita em Y em S , então $W_k(Y)$ de T_k também foi a última operação de escrita em Y em S'

76



Caracterizando Planos baseados na Seriabilidade

- Visão serializável
 - Definição de seriabilidade baseado em equivalência de visão
 - Um plano de execução é visão serializável se ele for visão equivalente a um plano de execução serial

77



Caracterizando Planos baseados na Seriabilidade

- Conflito Seriabilidade vs Visão Seriabilidade
 - Ambos são iguais se a hipótese abaixo for satisfeita
 - Hipótese da gravação restrita
 - Assume que se T escreve X, ela é determinada pelo valor de X que ela leu ($new\ X = f(old\ X)$)
 - Sob esta condição, ambas as definições são equivalentes

78



Caracterizando Planos baseados na Seriabilidade

- Conflito Seriabilidade vs Visão Seriabilidade
 - Hipótese da gravação irrestrita (cega)
 - O valor gravado em X por T, independe do valor antigo de X
 - $S_g: r_1(X); w_2(X); w_1(X); w_3(X); c_1; c_2; c_3;$ Gravações cegas
 - S_g é visão serializável ($T_1; T_2; T_3$)
 - S_g não é conflito serializável
 - Conflito serializável $\rightarrow$ Visão serializável

79



Caracterizando Planos baseados na Seriabilidade

- Conflito Seriabilidade vs Visão Seriabilidade
 - Teste de visão seriabilidade é NP

80



Outros Tipos de Equivalência de Planos

- Serialização de planos pode ser restritiva demais como forma de garantir a corretude
- O fato de não ser conflito serializável não implica que o plano esteja errado
- Sob certas circunstâncias semânticas, os planos de execução que não são conflito serializáveis podem funcionar corretamente

81



Outros Tipos de Equivalência de Planos

- Exemplo
 - Transação débito-crédito
 - Usando comutatividade de adição e subtração algumas transações não serializáveis podem funcionar corretamente

Transferência entre contas $T_1: r_1(X); X:=X-10; w_1(X); r_1(Y); Y:=Y+10; w_1(Y);$
 $T_2: r_2(Y); Y:=Y-20; w_2(Y); r_2(X); X:=X+20; w_2(X);$

Plano não-serializável

$S_h: r_1(X); w_1(X); r_2(Y); w_2(Y); r_1(Y); w_1(Y); r_2(X); w_2(X);$

Porém o plano está correto!!!

Débitos e créditos são comutativos.

82



Outros Tipos de Equivalência de Planos

- Pesquisas atuais
 - Ampliar a teoria de controle de concorrência de modo a tratar os casos em que a capacidade de serialização é considerada muito restritiva

83



Transações em SQL

- Conceitos similares aos vistos aqui
- Uma declaração SQL é considerada atômica
- Início da transação
 - Implícito: `SET AUTOCOMMIT=0;`
 - Explícito: `START TRANSACTION`
- Fim da transação
 - Explícito: `COMMIT` ou `ROLLBACK`

84



Transações em SQL

- Outras características podem ser especificadas usando (antes do início da transação)

SET TRANSACTION

– Modo de acesso

- **Padrão:** READ WRITE
- READ ONLY

– Escopo

- **Padrão:** aplica-se apenas à próxima transação
- GLOBAL
- SESSION

85



Transações em SQL

- Outras características podem ser especificadas usando (antes do início da transação)

SET TRANSACTION

– Nível de isolamento

- **Padrão:** REPEATABLE READ
- READ UNCOMMITTED
- READ COMMITTED
- SERIALIZABLE

86



Transações em SQL

- Sintaxe

- Modo de acesso

```
SET [GLOBAL | SESSION] TRANSACTION  
{ READ ONLY | READ WRITE }
```

- Nível de isolamento

```
SET [GLOBAL | SESSION] TRANSACTION  
ISOLATION LEVEL  
{ READ UNCOMMITTED  
  | READ COMMITTED  
  | REPEATABLE READ  
  | SERIALIZABLE }
```

87



Transações em SQL

- Com `SERIALIZABLE` a execução intercalada de transações respeitará nossa noção de seriabilidade
- Possíveis violações da seriabilidade nos outros níveis de isolamento
 - **Leitura de sujeira:** Leitura de um valor que foi escrito por uma transação que falhou (valor incorreto)
 - **Leitura não-repetível:** Permitindo que uma outra transação escreva um novo valor entre múltiplas leituras de uma transação (valores possivelmente diferentes para um mesmo item)
 - **Fantasma:** Novas linhas sendo lidas usando a mesma condição de leitura
 - Uma transação T_1 faz uma consulta ao BD (`SELECT`) que retornará uma tabela contendo as tuplas que satisfazem a busca. Se depois T_2 inserir uma nova tupla no BD e essa tupla satisfizer a condição, e T_1 for repetida, teremos um fantasma

88



Transações em SQL

Possíveis violações baseadas em isolamento
Níveis como definidos em SQL

Nível de Isolamento	Leitura Suja	Leitura Não-repetível	Leitura Fantasma
READ UNCOMMITTED	Sim	Sim	Sim
READ COMMITTED	Não	Sim	Sim
REPEATABLE READ	Não	Não	Sim
SERIALIZABLE	Não	Não	Não

89



Transações em MySQL

- (transactions.sql)

90



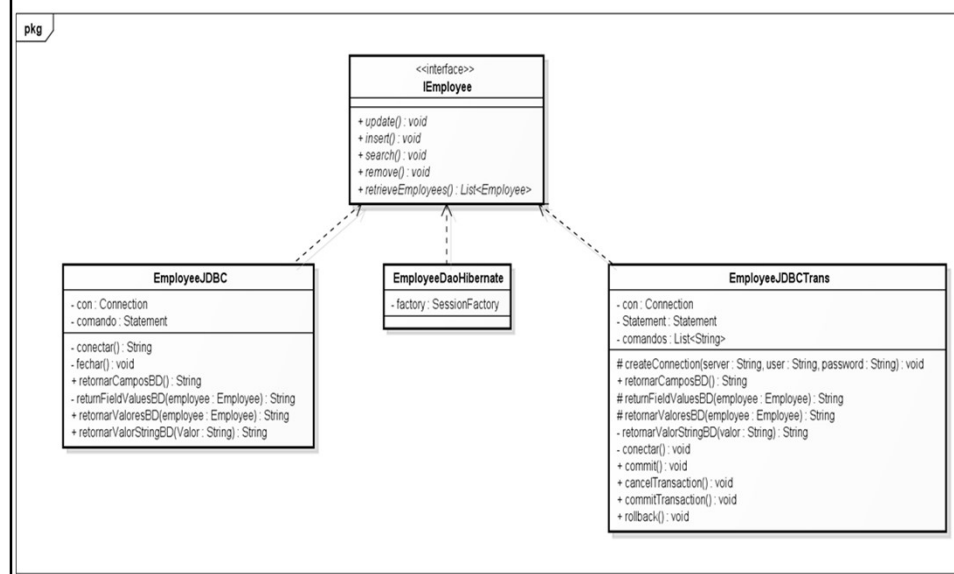
Transações em JDBC

```
conn.setAutoCommit(false);  
  
stat.executeUpdate(...);  
stat.executeUpdate(...);  
stat.executeUpdate(...);  
  
conn.commit();  
  
conn.rollback();
```

91



Transações em JDBC



92



Resumo

- Conceitos de transações
- Propriedades desejadas nas transações
 - ACID
- Caracterização de planos de execução baseados em restaurabilidade
- Caracterização de planos de execução baseados em seriabilidade
- Suporte SQL para transações
 - MySQL
 - JDBC

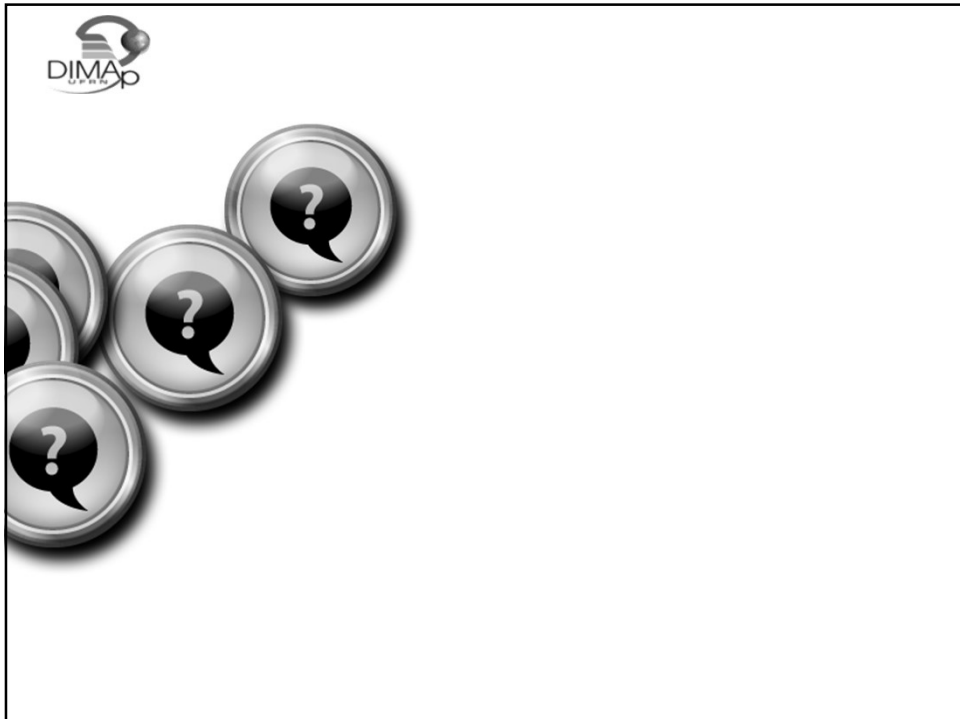
93



Leitura

- Capítulo 21

94



95